Cheffy AI Chatbot

Step-by-Step Build Process

Desi Dhamaka — AI Dining Concierge | July 8, 2026

This document describes everything built for Cheffy, the AI restaurant concierge on the Desi Dhamaka website. Work was delivered in modular phases so each layer could be tested and deployed independently. The public chat widget, Gemini backend, CMS knowledge, tool calling, personality engine, admin CMS, and Supabase configuration are all included.

Overview

- Assistant name: Cheffy — Your AI Restaurant Assistant (Powered by Chefgaa)
- Stack: React + TypeScript + Vite (frontend) · Netlify Functions (API) · Gemini (LLM) · Supabase (CMS + AI admin config)
- Locations: South Plainfield, Oak Tree (Edison), Lawrenceville — never mixed in one conversation
- Live site: https://desi-dhamaka-admin.netlify.app
- GitHub commit: 1e97fae (AI Concierge + Admin CMS module)
- Admin AI settings: /admin/integrations/ai-concierge

Architecture (high level)

Guest opens Cheffy !' React UI collects message !' CMS knowledge is loaded for the selected outlet !' Smart tools run based on intent !' enriched request goes to Netlify ai-concierge function !' Gemini streams reply !' UI parses actions, cards, and follow-up chips !' session memory persists in browser sessionStorage.

Step 1 — Project foundation & wiring

- Added Cheffy floating widget to all public location routes via AIProvider in App.tsx
- Created CheffyContext — single source for chat state, messages, knowledge, streaming
- Portal-based UI (Cheffy.tsx) — mascot + ChatWindow, does not block page content
- Location gate: Cheffy hidden until guest selects an outlet
- Files: src/components/ai/CheffyContext.tsx, Cheffy.tsx, AIProvider.tsx

Step 2 — AI provider layer (Gemini)

- Pluggable provider registry: gemini (active), openai/claude/mock (future-ready)
- GeminiProvider with streaming SSE support via Netlify function proxy
- Client never sees API keys — keys live in Netlify env (GEMINI_API_KEY)
- Configurable via VITE_AI_PROVIDER and admin provider settings
- Files: src/services/ai/provider.ts, providers/geminiProvider.ts, types.ts

Step 3 — Netlify server function (secure API)

- Created netlify/functions/ai-concierge.mts — production AI endpoint
- Accepts: message, history, cmsContext, toolResults, session, conversationId
- Returns: streaming SSE chunks or JSON fallback
- Builds system prompt server-side from CMS context + tool results
- Structured logging via lib/aiLogger.ts (no PII, no prompts in logs)
- CORS + method guards; service keys never exposed to browser

Step 4 — CMS knowledge layer

- Cheffy reads live Supabase CMS data per location — no hardcoded menu copy
- Modules: homepage, offers, gallery, reviews, SEO, restaurant settings (menu/reservations/ChefGaa future)
- buildCMSKnowledge(locationId) assembles a structured knowledge bundle
- Intent-based module query — only relevant CMS slices sent to the model
- CMS fallback replies when AI unavailable or still loading
- Files: src/services/cms/knowledge/*, src/services/ai/cmsKnowledge.ts

Step 5 — Smart tool calling

- Modular tool registry + executor under src/services/ai/tools/
- Intent resolver detects: menu, offers, hours, location, order, reservation, catering, etc.
- Handlers fetch CMS data and return structured tool results to the model

- Tool cache + logging for performance and observability
- Wired through enrichAIRequest() in context.ts — engine core untouched after integration
- Verify script: scripts/verify-cheffy-tools.ts

Step 6 — System prompt & configuration

- Canonical prompt in src/config/ai/systemPrompt.ts (CHEFFY_SYSTEM_PROMPT_CORE)
- buildCheffySystemPrompt() injects location, CMS context, tool results, session prefs
- Server mirror: netlify/functions/lib/cheffySystemPrompt.ts
- Rules: hospitality tone, no hallucinated prices, location isolation, action button format

Step 7 — Personality & hospitality engine

- Guest memory in sessionStorage — preferences, visit count, last topics
- Greeting engine: time-of-day, welcome-back, location-aware Namaste messages
- Emotion detection maps user intent !' mascot mood (thinking, celebrating, etc.)
- Conversation coach enriches system instructions for warm hosting
- Dynamic chips + follow-up suggestions based on topic and guest profile
- Recommendation architecture for family, veg, spicy, budget, celebration queries
- Files: src/services/ai/personality/*
- Verify script: scripts/verify-cheffy-personality.ts

Step 8 — Conversation engine & streaming

- Session memory: conversation history, conversation ID, entered flag, nudge timing
- Streaming via streamResponse() — token-by-token UI updates
- Action parsing: [BUTTON:Label|path] markup !' CheffyActionBar (order, map, phone, etc.)
- Presentation layer: recommendation cards, offer cards, contact cards
- Typing indicator + dynamic thinking messages based on detected intent
- Abort/cancel support when user sends new message mid-stream
- Files: src/services/conversation/*, src/components/ai/ChatWindow.tsx, MessageStreamRenderer.tsx

Step 9 — Chat UI (guest-facing)

- ChatHome dashboard — quick actions, popular topics, featured offer, recent chat
- Conversation view — message bubbles, streaming renderer, action buttons
- Quick action chips + follow-up chips after assistant replies
- Empty state with suggested questions when conversation is new
- Header: Cheffy · Your AI Restaurant Assistant · Powered by Chefgaa
- Mobile-responsive floating panel; footer-aware positioning
- Files: src/components/ai/ChatHome.tsx, ChatWindow.tsx, cheffy.css

Step 10 — Mascot animation (Cheffy character)

- GSAP-powered CheffyAnimator — transform-only motion for 60 FPS
- Homepage intro: hidden until outlet selected !' 2s delay !' peek !' greet !' float idle
- Session-once intro (sessionStorage cheffy_has_entered)
- Idle: breathing, floating, blinking, scarf sway, plate shimmer
- Outlet switch: slide off-screen + welcome bubble per location
- Hover: lift + cursor tracking + Need help? bubble
- Chat open: lean-forward focus pose; thinking status near mascot
- Celebrate on meaningful actions (order, directions, menu)
- prefers-reduced-motion + tab visibility pause respected
- Files: CheffyAnimator.ts, useCheffy.ts, CheffyMascotState.ts

Step 11 — AI Admin CMS module

- New sidebar item: Integrations !' AI Concierge (/admin/integrations/ai-concierge)
- Enterprise dashboard with 12 sections as cards:
- General, Personality, Knowledge, Providers, Prompt, Conversation,
- Suggestions, Analytics, Testing sandbox, Logs, Errors, Advanced, Location overrides
- CRUD services: src/services/aiAdmin/repository.ts
- Role permissions: Super Admin (full), Manager (prompts/greetings), Staff (read-only)
- Sandbox playground calls ai-concierge without modifying production data
- Log export CSV/JSON — no PII, prompts, or API keys stored
- Files: src/admin/pages/AIConciergePage.tsx, components/aiConcierge/*

Step 12 — Supabase database (migration 032)

- Pure SQL migration: supabase/migrations/032_ai_management.sql
- Tables: ai_settings, ai_personality, ai_provider_settings, ai_prompt_versions,
- ai_suggested_questions, ai_followups, ai_feature_flags,
- ai_conversation_logs, ai_tool_logs, ai_error_logs, ai_audit_log
- Row Level Security: admin-only via public.is_admin()
- Seed data: default settings, quick chips, follow-up suggestions
- Apply via Supabase SQL Editor or: supabase db push

- Important: run SQL only — never paste TypeScript into Supabase

Deployment process

Step 13 — Environment variables (Netlify)

- VITE_SUPABASE_URL + VITE_SUPABASE_ANON_KEY — browser CMS access
- GEMINI_API_KEY — server-side only in Netlify function
- VITE_AI_PROVIDER=gemini (optional, default gemini)
- Never commit .env — use .env.example as template

Step 14 — Build & deploy

- npm run build — TypeScript check + Vite production bundle
- npm run deploy — Netlify production deploy with functions
- Functions bundled: ai-concierge.mts (+ chefgaa-sync, analytics, etc.)
- Production URL: https://desi-dhamaka-admin.netlify.app
- GitHub: https://github.com/arunsaikumarB/dd3-restaurant-cms (commit 1e97fae)

Step 15 — Testing checklist

- Select location on gate page !' Cheffy appears after intro delay
- Ask about hours, offers, menu, directions — verify tool-backed answers
- Click action buttons (Order, Directions) — verify navigation
- Switch location in chat — verify context updates, no cross-location mix
- Admin: Integrations !' AI Concierge — save settings, run sandbox test
- Run: npx tsx scripts/verify-cheffy-tools.ts
- Run: npx tsx scripts/verify-cheffy-personality.ts
- npm run build — must pass with zero TypeScript errors

How admins configure Cheffy (no code)

- Log in at /admin !' Integrations !' AI Concierge
- General: enable/disable AI, floating assistant, memory, maintenance mode
- Personality: assistant name, tone, greetings, emoji level
- Knowledge: toggle which CMS modules Cheffy can read
- Prompt: edit system prompt with versioning and rollback
- Suggestions: reorder quick action chips and follow-up questions
- Testing: sandbox playground with latency and tool timeline
- Analytics & Logs: conversation stats, error logs, CSV export

Key file map

- Widget UI: src/components/ai/
- AI services: src/services/ai/
- CMS knowledge: src/services/cms/knowledge/
- Conversation: src/services/conversation/
- Admin module: src/admin/pages/AIConciergePage.tsx + src/services/aiAdmin/
- API: netlify/functions/ai-concierge.mts
- Database: supabase/migrations/032_ai_management.sql
- System prompt: src/config/ai/systemPrompt.ts

Future-ready (architecture supports)

- OpenAI and Claude providers (registry already wired)
- Menu, reservations, ChefGaa tools (feature flags + knowledge toggles)
- Multi-website reuse (ChefGaa, Logisoft, other clients)
- CSAT surveys and advanced analytics in admin

Summary: Cheffy is a production-ready AI concierge: Gemini-powered, CMS-aware, multi-location, with smart tools, hospitality personality, streaming chat, animated mascot, secure Netlify API, and a full admin CMS to configure everything without code changes.